

강지호

KANG JI HO

姜知昊

CONTACT

BORN

2001.09.11

PHONE

010-7666-4510

EMAIL

official@aiwebbuilder.kr

WEB

aiwebbuilder.kr

BASE

경기도 화성시 동탄구 메타폴리스

GITHUB

https://github.com/StockHedge

EDUCATION

경기대학교 경제학과

경제학 학사 · 졸업

2022.03 - 2026.02

FOCUS

AI Product

LLM Engineering

SaaS 운영

금융 · 퀀트

1인 개발

AI와 금융 필드에서 전천후로 활동하는 강지호입니다.

사회/경제 도메인 이해를 기반으로 LLM 엔지니어링을 결합해, 기획부터 개발·배포·운영까지
1인 체제로 SaaS 제품을 만들고 운영합니다.

EXPERIENCE 경력

- | | | |
|----|--|-------------------|
| 01 | StockHedge — 대표 · 1인 스타트업 | 2025.08 - 현재 |
| | AI 웹빌더 SaaS(aiwebbuilder.kr) 기획·개발·배포·운영 단독 수행 | |
| 02 | 민트투자자문 — AI Transformation | 2026.04 - 현재 |
| | 인턴 2026.04 - 2026.07 · 프리랜서 2026.07 - 현재
사내 통합 플랫폼 엔드투엔드 설계 및 LLM 엔지니어링 | |
| 03 | 토스페이 — Onboarding Operational Assistant | 2025.06 - 2025.08 |
| | 가맹점 온보딩 운영 프로세스 지원 및 BD 업무 | |

SELECTED WORK 주요 프로젝트

- | | | |
|----|--|-------------------|
| 01 | AIWebbuilder — 웹빌딩 플랫폼 | 2026.05 - 현재 |
| | 자연어 입력 기반 웹사이트 자동 생성 SaaS · 생성 품질/비용 최적화 | |
| 02 | 광고1동 상권 활성화 프로젝트 — 기획 / 총괄 | 2025.09 - 2025.12 |
| | 상권 데이터 분석 기반 실행안 수립 및 이해관계자 조율 | |
| 03 | 2025 수원 대학생 청년 정책 포럼 — 기획 / 총괄 | 2025.11 |
| | 포럼 전체 기획, 연사 섭외, 당일 운영 총괄 | |

ACTIVITIES & AWARDS 대외활동 · 공모전

- | | | |
|----|--------------------------------------|-------------------|
| 01 | 경기대학교 금융경제동아리 KFI — 초대 회장 | 2025.08 - 2025.12 |
| 02 | 제10회 DB GAPS 투자대회 | 2025.08 - 2025.12 |
| 03 | TV조선 서포터즈 | 2025.02 - 2025.04 |
| 04 | 2024 한국지능정보사회진흥원 데이터 크리에이터 대회 | 2024.09 - 2024.11 |
| 05 | 2024 한국발명진흥회 캠퍼스 특허 유니버시아드 | 2024.03 - 2024.07 |

REFERENCE (제목 클릭하시면 해당 링크로 넘어갑니다!) · 출판작 · 포트폴리오

- | | | |
|----|-------------------------------------|------------------|
| 01 | DB보험금융공모전 — VIX 레짐 원자재 선물 전략 | docs.google.com |
| 02 | 제10회 DB GAPS 투자대회 — 리서치 보고서 | docs.google.com |
| 03 | 금융경제분석 기말 연구 보고서 | drive.google.com |
| 04 | AIWebbuilder — AI 웹빌딩 플랫폼 | aiwebbuilder.kr |

— SOLO PRODUCT ENGINEERING · 2026

AI가 웹사이트를 만들고, 직접 배포까지 합니다

한국 소상공인이 품을 채우면 멀티에이전트 파이프라인이 단일 HTML을 생성하고 Cloudflare에 실제 URL로 배포하는 SaaS입니다. 기획·백엔드·프론트엔드·모바일 앱·인프라·결제 연동·운영까지 혼자 만들었고, 지금도 운영 중입니다.

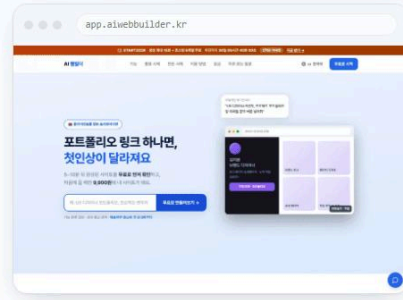
운영 중인 서비스

1인 개발·기획·운영

471 commits

라이브 서비스 열기 →

엔지니어링 판단 보기



— OVERVIEW

템플릿을 고르는 대신, 설명하면 사이트가 나옵니다

기존 노코드 빌더는 사용자가 직접 레이아웃을 조립해야 합니다. 이 서비스는 업종별 구조화 품만 채우면 AI가 설계·코딩·검수를 거쳐 완성된 단일 HTML을 만들고, 결제 후 실제 URL로 배포합니다. 미리보기는 무료이고 배포 시점에 과금하는 try-before-pay 구조입니다.

전체 코드

76,788 줄

백엔드 33,302 · 프론트 39,233 · 앱 4,253

백엔드 테스트

908 개

CI 게이트 · 실패 시 머지 차단

API 라우터 / DB 모델

26 / 29

마이그레이션 50개

생성 1건 원가

\$0.284

운영 30일 실적 평균

— SIGNATURE · AI PIPELINE

4단계 멀티에이전트 파이프라인

단계마다 요구되는 능력이 다르므로 모델을 분리했습니다. 정형화·검증처럼 판단 폭이 좁은 단계는 저비용 모델로 고정하고, 설계·코드 생성에만 상위 모델을 씁니다. 아래 수치는 추정치 아니라 `pipeline_runs.metrics` 에 적재된 실측값입니다.

STAGE 1

요구사항 정형화

품 응답을 JSON 스펙으로 변환. 프롬프트 인젝션·유해 입력은 해당 부분만 무시하고 합법 요청만 통과시키는 가드레일을 둡니다.

claude-haiku-4.5

STAGE 2

구조 설계

색선 구성·컬러 팔레트·레이아웃을 명세로만 출력합니다. 코드를 만들지 않아 설계와 구현의 책임이 분리됩니다.

claude-opus-4.8 · effort high

STAGE 3 · 최대 비용

코드 생성

단일 HTML/CSS/JS를 생성합니다. 모바일 우선 반응형, 터치 타겟 44px, OKLCH 변수 기반 테마 토권을 강제합니다.

claude-sonnet-4.6

19,733 76%
출력 토큰 비용 비중

STAGE 4

품질 검증

시맨틱 태그·변응형·빈 placeholder를 점검하고, 정적 검사가 잡아낸 사용자 입력 누락 을 원문 그대로 복원합니다.

claude-haiku-4.5

단계별 비용 비중 — Opus 경로 실측

총 \$0.507 · 272초

53 · 76%

52 · 19%

51-54

— ENGINEERING DECISIONS

측정하고, 진단하고, 판단한 기록

기능을 붙인 것보다 중요한 건 무엇을 근거로 결정했는가입니다. 아래 네 건은 모두 가설을 세워 예측한 뒤 수치로 결론을 낸 사례입니다.

01 AI가 사용자 입력을 조용히 버리던 문제		
증상 생성된 사이트에서 사용자가 입력한 메뉴 가격이 동파로 사라졌습니다 . 에러도 경고도 없이 그릴듯한 페이지가 나왔습니다.	진단 Stage 1의 출력 스키마에 그 값을 담을 슬롯이 없었습니다. 값이 살아남으려면 모델이 자유 텍스트 필드에 우연히 받아적어야 했고, 실측 결과 가격 100%·메뉴 50%가 유실 했습니다. 같은 모델인데 결과가 갈린 건 비결정성 때문이었습니다.	조치와 결과 요약 단계를 우회해 원문을 Stage 3에 직접 전달하고, 누락 검출기를 붙여 Stage 4가 복원하도록 했습니다. 메뉴·가격 3/3 복원. 비용은 오히려 10% 감소 했습니다 — 모델이 빈칸을 지어낼 필요가 없어서 출력이 17% 줄었기 때문입니다.
02 프롬프트 캐싱이 이득인지 손해인지 계산하기		
관측 캐싱을 걸어줬는데 전 스테이지에서 <code>cache_read = 0</code> 이었습니다. 절감이 전혀 없었습니다.	가설 배제 "배선이 잘못됐다"가 첫 가설이었지만 <code>cache_write</code> 는 정상 발생하고 있었습니다(S2 7,694 · S3 16,735). 배선은 멀쩡했고 재사용이 없었을 뿐 입니다 — 캐시 TTL 5분 안에 다음 생성이 들어올 만큼 트래픽이 촘촘하지 않았습니다.	판단 <code>write</code> 는 입력 정가의 1.25배라 손날비는 전액이 아니라 25% 프리미엄 뿐, 건당 \$0.022(총액의 4.4%) 로 정량화했습니다. 트래픽이 촘촘해지면 <code>read</code> 는 입력가의 1/10이라 즉시 이득으로 뒤집히고 되돌리기도 쉬운 결정이라 유지 를 택했습니다.
03 "더 싼 모델"이 정말 더 나은지 분리 계측		
실험 동일한 품 입력 으로 Opus 경로와 Gemini 경로를 각각 실행해 스테이지별 토큰·비용·소요 시간을 비교했습니다.	표면 결과 비용 -54% (\$0.507 → \$0.234), 소요 -51% (272초 → 133초). 원가율은 7.0%에서 3.2%로 떨어졌습니다. 여기서 멈췄다면 즉시 전환했을 겁니다.	한 겹 더 같은 출력 토큰을 Gemini 단가로 환산해보니 절감의 45%는 단가가 아니라 "결과물"이 28% 작아서 였습니다 (완성 HTML 20,379 → 14,614 토큰). "싸다"와 "효율적이다"는 다르다고 보고 전환을 보류했습니다 — 남은 판단은 비용표가 아니라 품질 검토의 몫입니다.
04 CI 요금이 예상보다 37% 많이 나온 이유		
불일치 실제 실행 시간은 1,247 분인데 청구는 1,968 분이었습니다.	원인 GitHub Actions는 워크플로 실행이 아니라 job 단위 로 분을 올림 처리합니다. 21초짜리 job도 2분으로 청구됩니다. 차이의 37%가 전부 반올림이었고, 짧은 job 을 여럿 두는 설계가 가장 비쌌습니다.	이런 HTTP 호출만 하는 크론을 Cloudflare Worker로 옮기고, 실연산이 필요한 DB 백업만 Actions에 남겼습니다. 덤으로 헬스체크 주기가 실측 45~80 분에서 15 분으로 개선됐습니다 — Actions의 스케줄 <code>throttle</code> 이 원인이었습니다.

— ARCHITECTURE

운영 인프라

사용자가 만든 사이트를 실제로 호스팅해야 하므로 생성 파이프라인과 배포 대상이 분리된 구조입니다. 지연을 줄이려고 컴퓨트는 도쿄, 데이터베이스는 싱가포르 리전에 두었습니다.

APPLICATION	DATA & JOBS	EXTERNAL
프론트엔드 Cloudflare Pages	Postgres Neon Singapore · Pooled	결제 (국내) KakaoPay 단건 · 정기
백엔드 API Fly.io Tokyo · Dedicated IPv4	Redis Upstash Tokyo · TLS	결제 (해외) Creem MoR · 웹훅 HPAAC
모바일 앱 Expo SDK 57 · expo-router	오브젝트 스토리지 Cloudflare R2 · 썸네일	알림 CoolSMS · Resend · Telegram
사용자 사이트 site-{uuid}.pages.dev	스케줄러 Cloudflare Workers	생성 모델 Claude · Gemini

— TECH STACK

개발 스펙

타입 안정성과 비동기 처리를 기본값으로 두고, 결제·인증처럼 되돌리기 어려운 경로에는 멍등성 검증을 강제했습니다.

Backend

Python 3.11 FastAPI SQLAlchemy 2.0 async asyncpg Alembic
Pydantic v2 httpx pytest ruff

Frontend

React 18 TypeScript strict Vite 5 TanStack Query Zustand
React Hook Form Zod Tailwind i18n 5개 언어

AI Pipeline

Claude Opus 4.8 Claude Sonnet 4.6 Claude Haiku 4.5 Gemini 3.1 Pro
prompt caching 토큰·비용 계획 입력 정화

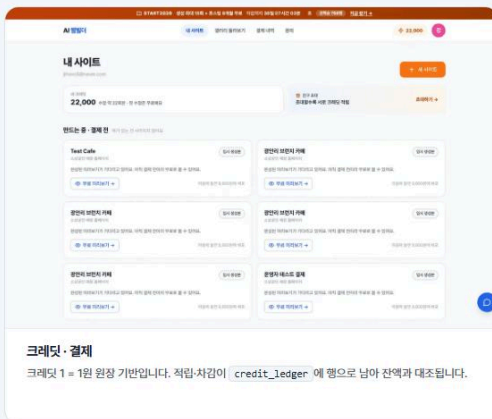
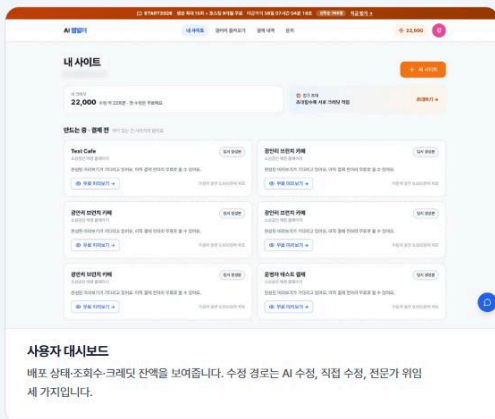
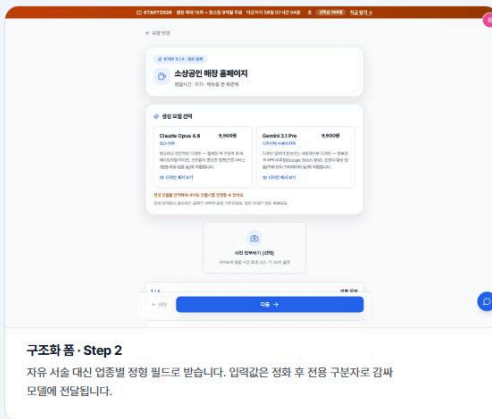
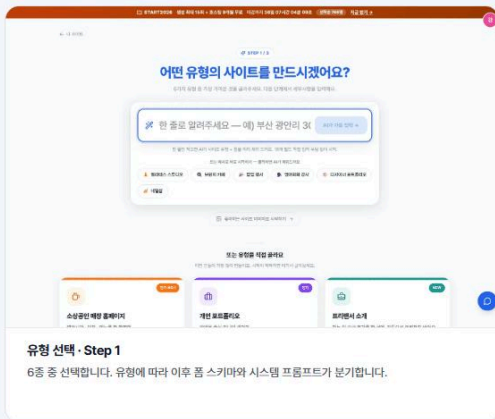
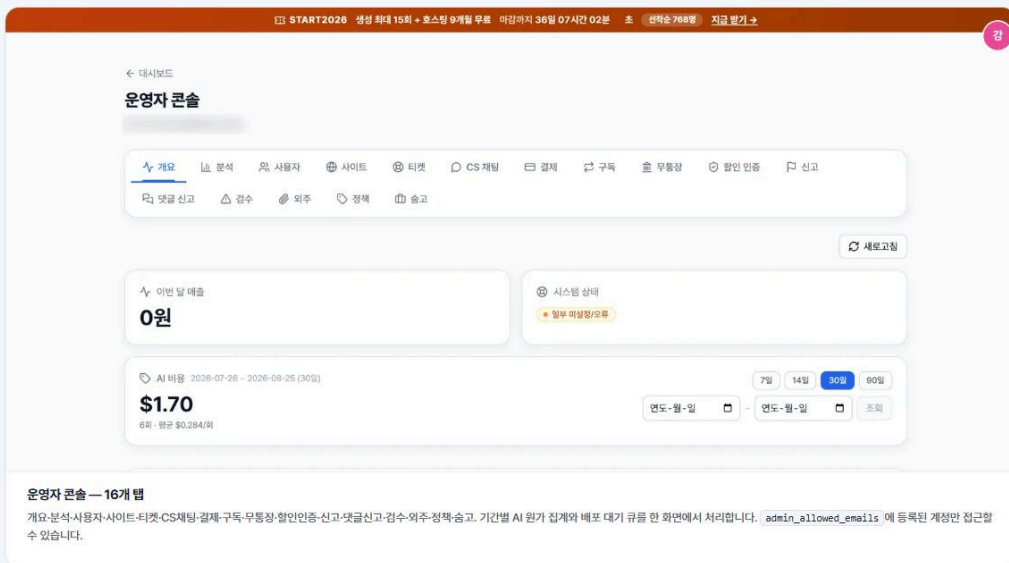
Infra & Ops

Cloudflare Pages · R2 · Workers Fly.io Neon Postgres
Upstash Redis Github Actions CI Sentry Docker

PRODUCT SCREENS

실제 서비스 화면

모두 운영 중인 서비스에서 직접 캡처한 화면입니다. 가입자 개인정보가 보이는 영역은 가렸습니다.





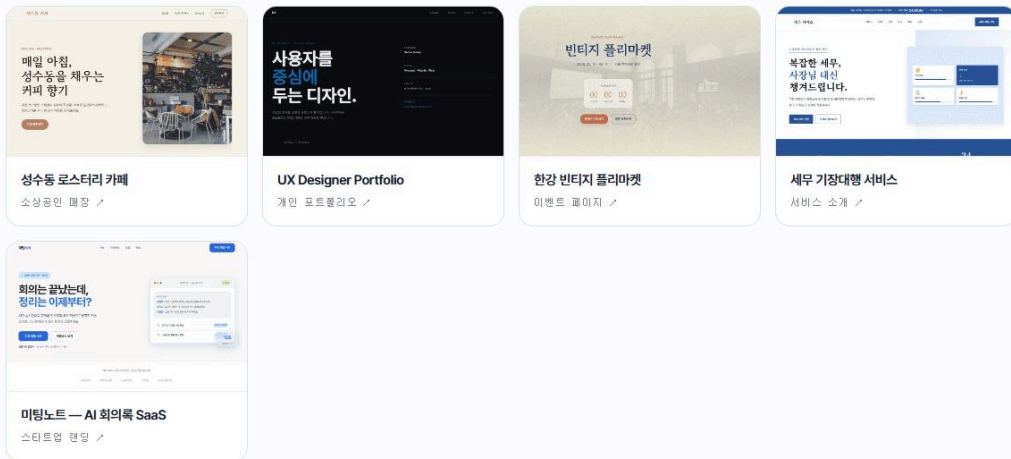
갤러리 — 사용자가 만든 사이트

공개 설정한 사이트가 모이는 곳입니다. 조회 좋아요댓글과 신고 처리 경로가 붙어 있습니다.

PIPELINE OUTPUT

파이프라인이 실제로 만든 사이트

아래는 설명이 아니라 산출물입니다. 모두 이 파이프라인이 생성해 배포한 실제 URL이며, 지금 클릭해서 열어볼 수 있습니다.



BEYOND THE HAPPY PATH

운영하면서 풀어야 했던 것들

기능이 도는 것과 서비스가 도는 것은 다릅니다. 아래는 정상 경로 바깥에서 마주친 문제들입니다.

생성물을 그대로 배포할 수 없다

LLM 산출물을 무검증 배포하면 빈 페이지나 `eval` 이 섞인 HTML이 사용자 도메인에 올라갑니다. 배포 직전 게이트를 2단계로 나눠 구조 검증-위험 스크립트는 차단하고, 리소스 호스트-용량 문제는 경고만 남깁니다. 사용자의 지도-SNS 링크까지 막지 않도록 `!a href` 는 검사 대상에서 제외했습니다.

결제는 두 번 눌러도 한 번이어야 한다

결제 콜백과 OAuth 콜백은 React StrictMode의 이중 호출과 사용자의 세로고침 양쪽에 노출됩니다. 프론트에는 `sessionStorage` 먹등성 키를, 백엔드 웹훅에는 서명 검증과 상태 전이 검사를 걸어 중복 승인을 차단했습니다.

남의 번호로 문자를 보내면 안 된다

문의 알림 SMS 수신번호를 자유 입력으로 두면 타인 번호로 스팸을 보내고 요금까지 전가할 수 있습니다. 가입 인증한 본인 번호는 즉시 저장하되, 다른 번호는 OTP 통과 후에만 저장하도록 막았습니다.

CONTACT

강지호

이메일 jihono55@gmail.com

GitHub github.com/StockHedge

기획부터 운영까지 제품 하나를 끝까지 책임지는 방식으로 일합니다.
코드와 운영 기록 전체를 함께 보실 수 있습니다.

서비스

app.aiwebbuilder.kr

이 문서의 모든 수치는 저장소와 운영 데이터베이스에서 직접 측정된 값입니다. 코드 규모는 git 및 소스 집계, 파이프라인 비용-소요 시간은 pipeline_runs.metrics 실측치, 확연은 운영 중인 서비스의 실제 캠퍼입니다.
저장소는 private이므로 코드 열람이 필요하시면 이메일로 요청해 주세요.

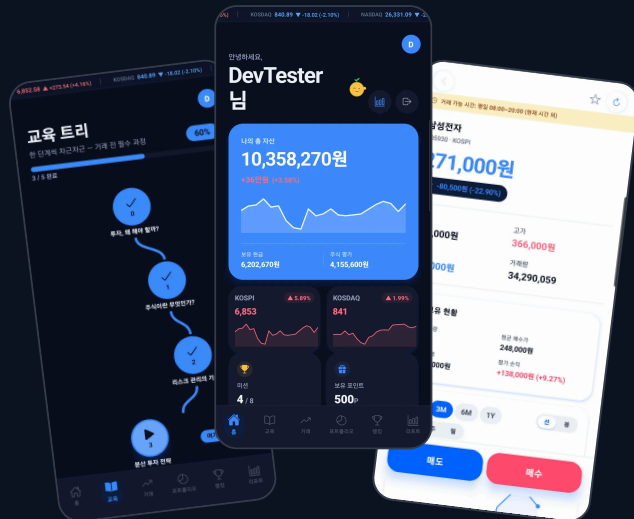


FinPle · 핀쉐어 FinShare

청소년 모의투자 플랫폼

투자, 배움부터.

만 14세 이상 청소년이 실제 돈을 잃지 않고 투자를 배우는 모의투자·금융교육 앱. 기획·백엔드·앱·인프라·QA 전 영역을 혼자 만들어 운영 배포까지 마쳤습니다.



기간

2026.03 – 2026.08

역할

1인 개발 (Full-stack)

상태

● 운영 배포 중

CODE

29,714 줄

COMMITTS

162

MERGED PR

73

API

80 엔드포인트

MODELS

25 테이블

MIGRATIONS

26

SCREENS

25

PYTEST

195 PASS

TSC

0 ERROR

OVERVIEW

교육을 통과해야 거래가 열린다

모의투자를 교육의 보상이 아니라 교육의 연장선에 두기 위해, 제품 구조와 체결 규칙을 그렇게 설계했습니다.

청소년 금융교육은 대체로 **읽고 끝난다**. 반대로 실전 투자 앱은 미성년자에게 열려 있지 않고, 열려 있더라도 손실 위험을 그대로 진다. 배움과 실행 사이가 비어 있다.

FinPle은 그 사이를 메운다 — 배운 것을 실제 시장 데이터 위에서 위험 없이 실행해 보게 한다.

교육 트리(Level 0-20)를 통과하면 거래가 열리고, 체결은 **실제 KRX 종목 2,770개**의 시세로 돈다. 수수료 0.015%, 매도 거래세 0.23%, 슬리피지 허용 0.5%, 주문 가능 시간까지 실제와 같은 규칙을 적용해 '장난감 계좌'와 구분했다.

투자 결과가 아니라 **투자 습관**을 보상한다. 수익률 보상 미션은 은퇴시키고 최대낙폭 관리·분산 보유 미션으로 대체해, 수익률을 쫓는 유인을 제품에서 걷어냈다.

기간	2026.03.15 – 2026.08.24 약 5개월 · 커밋 162건
역할	1인 개발 기획 · 백엔드 · 앱 · 인프라 · QA
대상	만 14세 이상 청소년 중·고교 5,704곳 데이터 내장
백엔드	finple-backend.fly.dev Fly.io nrt · 운영 중
웹	finple.pages.dev Cloudflare Pages
저장소	StockHedge/YouthInvestment GitHub · private

ARCHITECTURE

시스템 구성

단일 FastAPI 서비스가 도메인 로직을 전담하고, 외부 시세·인증·발송은 전부 실패해도 서비스가 죽지 않도록 graceful degrade 시켰습니다.



기술 스펙

백엔드		프론트엔드	
FastAPI	0.111	Expo SDK	55
SQLAlchemy	2.x async	React Native	0.83.6
Alembic	마이그 26	React	19
Pydantic	스키마 검증	TypeScript	5.9 · 0 error
PyJWT	2.10.1	react-native-svg	차트 · 마스코트
bcrypt	4.2.1	safe-area-context	edge-to-edge
asyncpg	async 드라이버	다크 모드	전 화면 대응
Python	3.11		
데이터 · 인프라		품질	
PostgreSQL	15 · Neon	pytest	195 passed
SQLite	테스트 자동 분기	asyncio_mode	auto
Redis	7	tsc strict	0 error
Fly.io	nrt (도쿄)	실기기 QA	Galaxy Note20
Cloudflare Pages	웹 배포	운영 E2E	전 플로우 완주
GitHub Actions	배포 · EOD 수집	uiautomator2	QA 자동화
Docker Compose	로컬 스택	gitleaks	시크릿 차단

SCREENS

실제 앱 화면

Galaxy Note20 실기기와 에뮬레이터에서 운영 백엔드에 붙여 캡처했습니다. 라이트·다크 모드를 전 화면에서 대응합니다.



홈 — 자산 요약

총자산·현금·주식 평가를 한 카드에 묶고, 지수 위젯은 전일 종가 대비로 티커바와 기준을 통일했습니다.

숫자 카운트업 · 실시간 지수



교육 트리

Level 0~20을 지그재그 노드로 배치해 진행 상태를 한눈에 보여줍니다. 퀴즈는 서버에서 자동 채점하고 XP를 적립합니다.

자동 채점 · XP · 진행률



종목 상세

시세·보유 현황·차트를 한 화면에 담고, 주문 가능 시간 밖이면 상단에 사유를 먼저 알립니다.

MA5/20/60 · 볼린저 · 캔들



차트 · 호가

차트 크로스헤어는 웹 DOM API 대신 RN responder로 구현해 네이티브에서도 동작합니다. 호가를 탭하면 지정가로 들어갑니다.

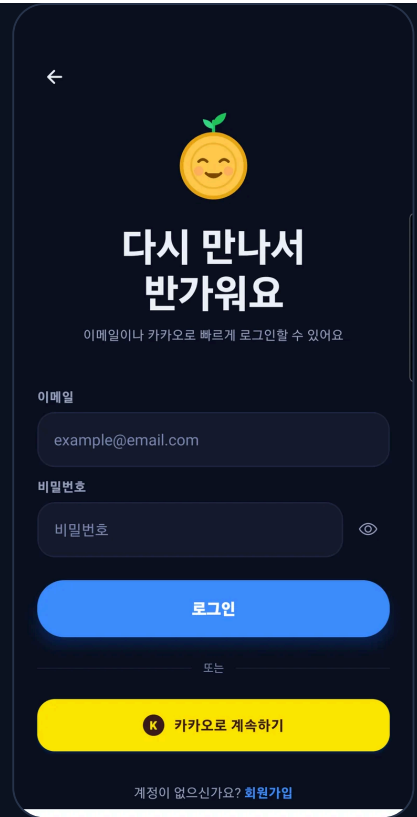
크로스헤어 · 지정가 연동



마이페이지

포인트·배지·기프트콘 교환·예적금 시뮬레이션을 한 곳에 모았습니다. API 실패는 0원이 아니라 - 와 복구 안내로 구분합니다.

포인트 원장 · 기프트콘 교환



로그인

이메일과 카카오톡 OAuth 두 경로. 액세스 토큰 만료는 인터셉터가 401을 잡아 재발급 후 원 요청을 재시도합니다.

JWT · 카카오톡 OAuth

핵심 구현

모의투자라도 규칙이 실제와 다르면 배움이 왜곡됩니다. 체결·시세·보상·랭킹 네 축을 실제 시장 제약에 맞췄습니다.

01 ORDER

주문 체결 엔진

시장가·지정가 주문을 받아 수수료와 세금을 실제 효율로 차감하고, 표시가와 체결가의 괴리가 허용치를 넘으면 체결을 거절합니다. 주문 가능 시간은 **fail-closed**가 기본값이라 설정이 비어 있으면 거래가 막히는 쪽으로 떨어집니다.

정합성 검증은 눈이 아니라 계산으로 했습니다 — 운영 계정으로 매수 5건을 체결한 뒤 수수료 372원을 포함한 총자산 9,994,628원이 장부와 일치하는지 계산했습니다.

수수료	거래세	슬리피지
0.015%	0.23% 매도	0.5%
주문 창		
08:00-20:00		

02 MARKET DATA

4단 폴백 시세 파이프라인

외부 시세는 언제든 끊깁니다. 그래서 단계마다 다음 소스로 떨어지도록 만들었습니다 — `kis:price` 실시간 틱 → KIS REST 현재가 → `price_history` 최신 종가 → `base_price`. 어느 단계에서 왔는지를 **가격 출처(실시간/종가)**로 화면에 명시합니다.

멀티 워커에서 WebSocket이 중복 구독되지 않도록 Redis 리더 락 `kis:ws:leader` 로 단일 스트림을 보장하고, 10초 TTL 캐시로 표시가와 체결가가 벌어지지 않게 고정합니다. 종가는 GitHub Actions가 평일 16:30 KST에 수집합니다.

종목	캐시 TTL	EOD 수집
2,770	10s	평일 16:30

03 GAMIFICATION

미션·포인트·기프트콘

거래·학습 이벤트가 발생하면 미션 엔진이 조건을 재평가하고, 달성분은 포인트 원장에 적립합니다. 포인트는 기프트콘으로 교환되고, 운영자는 신청을 **취소·전액 환불**할 수 있습니다. 재고는 `-1 = 무제한`, `0 = 품절` 계약으로 분리했습니다.

보상 설계 자체를 한 번 갈아엎었습니다. 수익률 보상 미션은 은퇴시키고 최대낙폭 15% 이내 유자·3종목 동시 보유 같은 **위험관리 미션으로 대체**했습니다.

미션	교환
7종 + 주간	취소·환불 지원

04 RANKING

스케줄러 없는 월간 랭킹

랭킹을 '가입 후 누적'에서 **월간**으로 바꾸면 매달 1일 KST에 기준을 초기화해야 합니다. 초기화 배치를 두는 대신, 집계 시점에 이번 달 기준 NAV가 없는 사용자를 멍등하게 일괄 INSERT하도록 만들었습니다.

결과적으로 **스케줄러가 없어도 월 전환이 자동**이고, 캐시 키에 월을 넣어 지난달 캐시는 자연 무효화됩니다. 죽은 경로였던 종전 TWR 계산은 제거했습니다.

범위	학교
전국·교내	5,704곳

근본 원인까지 내려간 결함

증상을 가리는 대신 원인 계층까지 내려가 고친 사례입니다. 재현되지 않는 테스트 환경이 결함을 숨기고 있던 경우가 가장 많았습니다.

증상

모든 미션이 한 이벤트씩 늦게 달성 처리됨

원인

운영 세션이 `autoflush=False` 라 미션 평가 SELECT가 같은 트랜잭션의 pending 주문·포지션을 보지 못했다. 테스트 `conftest` 는 autoflush 기본값이라 결함이 재현되지 않았다.

조치

`_evaluate_codes` 진입 시 flush. 회귀 테스트를 **운영과 동일한 세션 설정으로 재작성**해 테스트가 결함을 잡도록 만들었다.

증상

운영 배포 후 미션 화면이 통째로 비어있음

원인

빈 DB 초기화 경로가 `create_all` + `alembic stamp head` 였다. stamp는 스키마를 '적용됨'으로 표시만 하고, 마이그레이션 안의 데이터 시드는 실행하지 않는다.

조치

시드 상수를 로드해 먹등 반영하는 `seed_data_sync` 를 release 커맨드에 넣었다. **스키마와 시드의 책임을 분리**한 것이 핵심.

증상

웹 배포판에서 아이콘이 전부 𐄂로 렌더됨

원인

`wrangler pages deploy` 가 `node_modules` 경로 자산을 업로드에서 제외 → 아이콘 `.ttf` 요청이 SPA fallback(`text/html`)으로 200을 받아 FontFace 파싱이 거부됐다. 200 응답이라 네트워크 탭만 보면 정상으로 보였다.

조치

배포 스크립트가 `assets/node_modules` 를 `assets/vendor` 로 rename하고 번들 경로를 치환하도록 수정. 배포 후 `font/ttf` Content-Type을 실측 확인했다.

증상

종목 검색에서 한 글자 칠 때마다 포커스와 키보드를 잃음

원인

검색창이 FlatList의 `ListHeaderComponent` 에 컴포넌트 함수로 들어가 있었다. 리렌더마다 새 타입으로 인식돼 헤더가 통째로 리마운트됐다.

조치

검색창을 리스트 밖 고정 영역으로 분리하고, 검색창 2개를 1개로 통합했다. 자동완성과 목록 필터를 한 입력이 담당한다.

증상

홀 총자산이 0원으로 고정되는 경우가 있음

원인

숫자 카운트업이 `requestAnimationFrame` 기반인데, 비활성 탭이나 백그라운드 앱에서는 rAF가 멈춘다. 애니메이션이 시작값 0에서 영구 정지했다.

조치

문서가 비활성이면 즉시 최종값을 적용하도록 변경. **중요한 정보가 애니메이션 완료 뒤에만 보이지 않게** 하는 원칙을 컴포넌트에 못 박았다.

기능을 덜어낸 결정들

사용자가 미성년자라는 조건이 기능 추가보다 기능 제거를 더 많이 요구했습니다.

ETF를 지원하지 않는다

레버리지·인버스까지 딸려 들어오는 위험 노출을 청소년에게 열지 않기로 했습니다. 화면에서 감추는 데 그치지 않고 `instruments` 테이블의 관련 컬럼까지 마이그레이션으로 제거했습니다.

수익률을 보상하지 않는다

수익률 미션은 필연적으로 몰빵을 유도합니다. 시즌 최대낙폭 15% 이내 유지, 3종목 동시 보유처럼 **위험을 관리했을 때** 보상이 나오도록 바꿨습니다.

거래 시간은 fail-closed

환경변수가 비어 있으면 거래가 열리는 게 아니라 **막히는** 쪽이 기본값입니다. 운영 설정 실수가 곧 통제 없는 거래로 이어지지 않게 했습니다.

외부 연동은 전부 선택적

SMS·뉴스·환율·푸시·운영자 알림 키가 없으면 예러 대신 해당 기능만 조용히 비활성화됩니다. 부가 기능의 장애가 거래와 학습을 막지 않습니다.

검증 결과

아래 수치는 문서에서 옳긴 값이 아니라 이 문서를 만들면서 저장소에서 다시 측정한 값입니다.

195

passed

pytest

테스트 파일 33개 · 실패 0건.
PostgreSQL 방언 대신 ORM 표현
으로 작성해 SQLite에서도 동일하
게 돈다.

0

error

TypeScript strict

앱 코드 16,995줄 · `tsc --noEmit` 기준.

0.11

%

Janky frames

Galaxy Note20 실기기
`gfxinfo` 실측 — 17,084프레임
중 18. 90th 13ms.

73

PR

머지된 Pull Request

커밋 162건 · 전부 squash 머지. 마
이그레이션 26개는 운영 DB에 리
허설 후 적용.

운영 E2E

운영 백엔드에 붙여 로그인 → 교육 퀴즈 → 투
자 성향 조사 → 매수 체결 → 포트폴리오 검산
→ 미션 클레임 → 기프티콘 교환까지 한 계정으로 완주. 이 과정에서 결함 6건을 찾아 원인까지 수정했다.

실기기 QA

Galaxy Note20(SM-N981N)에서
`uiautomator2`로 화면을 자동 순회하며 캡처.
검증. edge-to-edge 탭바 겹침, 티커바 상태
바 침범을 실기기에서 잡았다.

마이그레이션

운영 스키마를 임시 DB에 복제해 **업그레이드 리
허설을 먼저 돌린 뒤** 배포했다. release 커맨드
가 `alembic upgrade head`를 실행한다.

FinPle · 핀웨어 FinShare · 강지호

화면 캡처는 Galaxy Note20 실기기와 에뮬레이터에서 운영 백엔드에 연결해 촬
영했습니다. 수치는 2026-08-25 저장소 실측 기준입니다.

[웹 데모](#)[API 헬스체크](#)[GitHub](#)

PROJECT 01 · 사내 운영 시스템

마케팅 자동화 허브

두 개의 SaaS 플랫폼이 만드는 마케팅 활동을
수집 → 지능 → 게이트 → 실행 → 피드백 5계층으로
자동화하고, 하나의 대시보드에서 관제하는 시스템입니다.
복극성 지표는 좋아요가 아니라 가입·결제 전환입니다.

• 운영 중 · Fly.io v60

기간	2026.08 — 운영 중
역할	설계 · 구현 · 배포 · 운영 전 구간
규모	54,862 LOC · 177 커밋
대시보드	marketing-hub-dashboard-1lt.pages.dev
API	marketing-hub.fly.dev

879건

백엔드 테스트 통과
(프론트 vitest 73건 별도)

17개

자동 실행 배치 잡
6개 스케줄 레인

29개

DB 테이블
API 라우터 21개

3중

역등성 방어
+ 광고 예산 하드캡

OVERVIEW

왜 만들었는가

SaaS 두 개를 혼자 운영하면 마케팅은 가장 먼저 방치됩니다. 콘텐츠를 만들 시간도, 성과를 추적할 시간도 없고, 광고는 "잘못 눌러서 돈이 나가는" 리스크 때문에 아예 켜지 못합니다.

문제

- 콘텐츠 기획·제작·발행이 전부 수작업이라 주 1회도 유지되지 않음
- 좋아요·조회수는 보이는데 가입·결제로 이어졌는지는 알 수 없음
- 광고는 오조작 한 번이면 예산이 중발 — 그래서 켜지 못함
- 플랫폼이 둘이라 지표가 섞이고, 어느 쪽 성과인지 구분이 안 됨

해법

- 파이프라인화 — 수집부터 성과 측정까지 13단계 상태머신으로 자동 진행
- 전환 어트리뷰션 — UTM·프로모코드를 발급 시점에 콘텐츠와 묶어 가입·결제까지 추적
- 가드레일 — 예산 상한을 코드 상수로 못 박아 설정으로도 완화 불가
- 워크스페이스 차원 — 성과·리드·동의를 플랫폼별로 분리 저장

ARCHITECTURE

계층 구조

각 계층은 아래 계층만 알고 위 계층은 모릅니다.
예를 들어 실행 계층은 Meta 연동이 실제 API인지 스텝인지 알지 못한 채 동작합니다.

01

수집

collectors/

youtube

ad_library

business_discovery

hashtag

02

지능

intelligence/

bandit

extractor

generator

brand_kit

media

ad_engine

03

게이트

gates/

compliance · 롤 + LLM 2단

tiering

approval

04

실행

executors/

publisher

email_sender

ads_executor

community

pacing

05

피드백

rollup

platform_sync

watchers

bandit_update

report

콘텐츠 상태머신 — 13개 상태, 전이는 코드가 강제

draft



compliance_check



pending_approval



approved



scheduled



publishing



published



measured

compliance_failed

rejected

hold

publish_failed

dead_letter

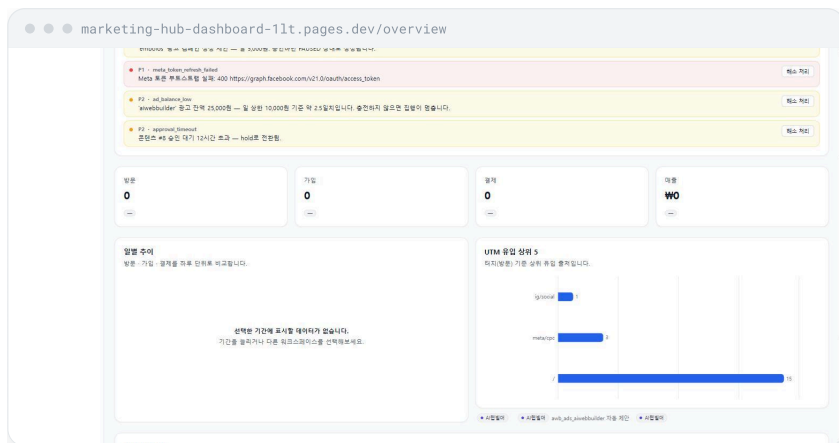
상태 컬럼에 값을 직접 대입할 수 없습니다. 모든 전이는 `assert_transition()` 을 거치고, 허용되지 않은 전이는 `ValueError` 로 즉시 실패합니다. `dead_letter` 는 재시도 예산 소진 지점이지 콘텐츠 결함이 아니므로, 외부 원인(권한·토큰)이 해소되면 같은 캠페인-미디어로 되살릴 경로를 남겨뒀습니다 — 실제로 Instagram 관한 부족으로 발행이 3회 소진돼 DB를 직접 만질 뻔한 사고에서 나온 설계입니다.

SCREENS

실제 운영 화면

아래는 목업이 아니라 현재 운영 중인 대시보드를 그대로 캡처한 것입니다.

데이터·시각·상태는 캡처 시점(2026-08-25)의 실제 값입니다.



01 / OVERVIEW

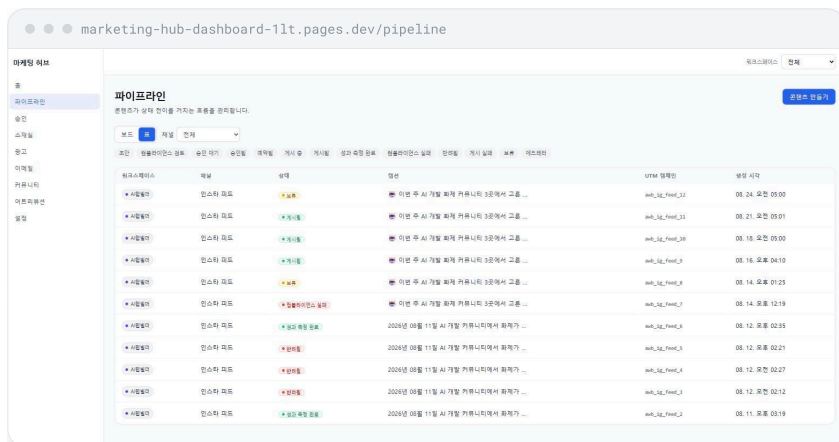
관제 홈

워크스페이스별 획득 지표를 한 화면에 모읍니다. 종ಾಯ가 아니라 방문 → 가입 → 결제 → 매출을 같은 축에 놓아, 실제 전환을 만들었는지로만 판단하게 했습니다.

기간 7 / 30 / 90일 토글

차트 일별 추이 · UTM 유입 상위 5

빈 상태 "데이터 없음" 대신 다음 행동을 제시



02 / PIPELINE

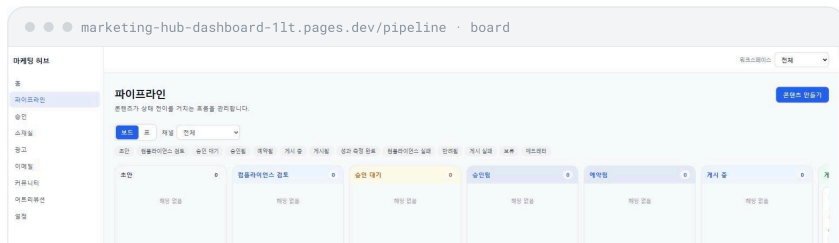
콘텐츠 파이프라인

상태머신 13단계가 그대로 UI 가 됩니다. 보류 · 게시됨 · 컴플라이언스 실패 · 반려됨이 색으로 구분되고, 각 건에 utm_campaign 이 붙어 전환을 역추적합니다.

뷰 캠페인 보드 / 표 전환

필터 워크스페이스 · 채널 · 상태 13종

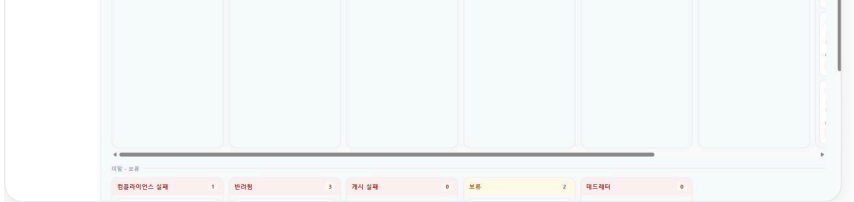
추적 개별 UTM 캠페인 자동 발급



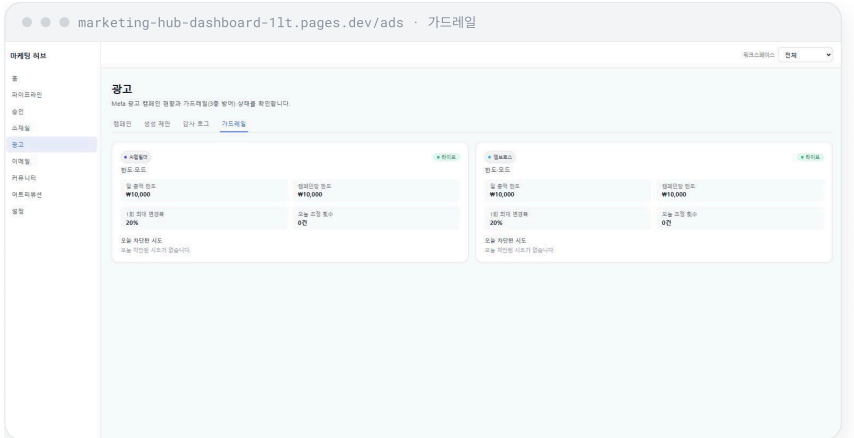
03 / KANBAN

흐름과 이탈의 분리

정상 흐름은 위쪽 가로 트랙, 이탈·보류는 아래쪽 별도 트랙으로 뱌었습니다. 막힌 것이 정상 흐름에 섞여 보이면 무엇이 멈춘는지 한눈에 못 찾기 때문입니다.



상단 초안 → 게시 중 정상 경로
하단 컴플라이언스 실패 · 반려
보류 · 데드레터

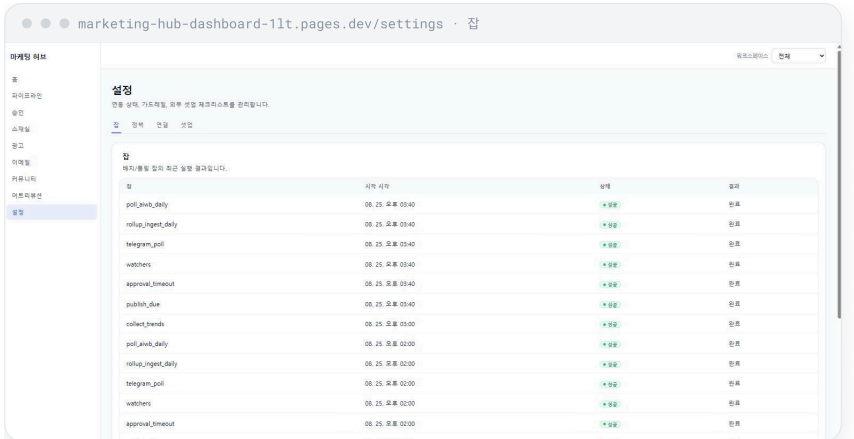


04 / GUARDRAIL

돈이 나가는 경로의 한도

광고는 오작작 하나로 예산이
증발합니다. 그래서 상한을
코드 상수로 두고, 운영 설정은
더 엄격한 방향으로만
반영되게 했습니다.

하드캡 일 30,000원 / 월 300,000원
캠페인 15,000원
변경폭 1회 최대 20%
증액 이 화면에서 불가
텔레그램 승인(T2) 필수

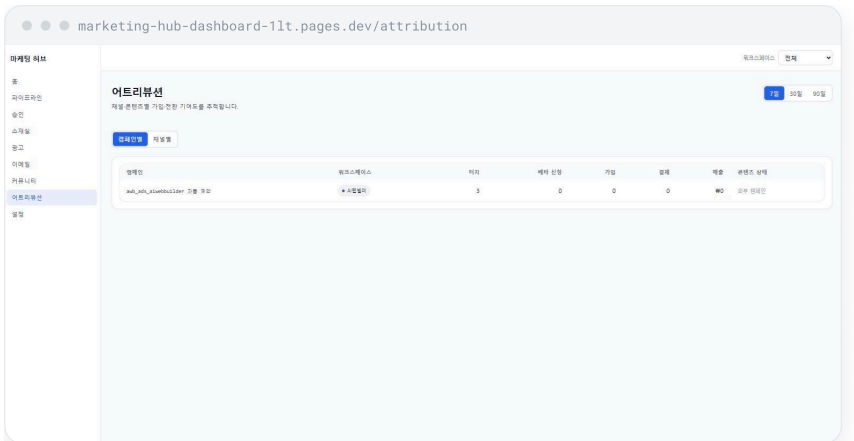


05 / JOBS

배치 실행 이력

17개 잡이 6개 레인에서 돌고,
결과는 전부 job_runs 에
남습니다. 같은 잡이 겹치지
않도록 advisory lock 으로
감싸고, 건별로는
SKIP LOCKED 를 씁니다.

100분 publish_due ·
approval_timeout · watchers
야간 토근 회전 · 밴딧 갱신
미디어 비공개화
기록 성공·실패·소요·오류
(시크릿 제거 후)



06 / ATTRIBUTION

전환 기여도 추적

캠페인 한 줄에 터치 → 베타
신청 → 가입 → 결제 → 매출이
이어집니다. GA4 없이 자체
이벤트로 구현했는데, UTM →
가입 조인은 GA4 로는
불가능하기 때문입니다.

속 캠페인별 / 채널별 전환
연결 프로모코드 발급 시점에
콘텐츠와 바인딩

ENGINEERING

핵심 설계 결정

"돌아가게 만드는 것"과 "혼자 운영해도 안 터지게 만드는 것"은 다릅니다.
아래는 후자를 위해 코드 레벨에 못 박은 불변조건들입니다.

SAFETY

가드레일은 완하 불가

CONSISTENCY

어휘 단일 소스

예산 상한을 guardrails/limits.py 코드 상수로 두고, 런타임 설정은 min(코드상수, 설정) 으로만 반영합니다. DB 를 고쳐도 상한은 내려가지 않습니다.

왜: 운영 설정이 안전장치를 무력화할 수 있으면 그런 안전장치가 아닙니다. 완화하려면 배포가 필요하게 했습니다.

채널·상태·티어·액션 어휘를 models/vocab.py 한 곳에만 정의하고, STATUS_TRANSITIONS 가 상태머신을 강제합니다.

왜: 상태 문자열이 흩어지면 오타 하나가 조용히 흐름을 끊습니다. 어휘를 하나로 묶어 일찍 잡습니다.

IDEMPOTENCY

역등성 3중 방어

- ① advisory lock 으로 잡 중복 실행 차단
- ② 자연키 ON CONFLICT 는 누적이 아닌 대입
- ③ 건별 트랜잭션 + FOR UPDATE SKIP LOCKED

왜: 크론은 재시도되고 겹칩니다. 한 겹만 두면 "같은 글이 두 번 올라가는" 사고가 언젠가 납니다.

TESTABILITY

외부 연동 4모드

Meta 연동은 disabled / stub / dryrun / live 네 모드를 갖고, 상위 계층은 모드를 모릅니다. 토큰이 없어도 전 구간이 완주됩니다.

왜: 외부 API 가 없으면 테스트가 막히는 구조는 결국 테스트를 안 쓰게 만듭니다.

BLAST RADIUS

부작용 초크 포인트 3개

외부로 나가는 모든 행위는 send_email() · apply_ad_action() · publish() 세 함수만 통과합니다. 다른 경로는 금지입니다.

왜: 검증 지점이 흩어지면 언젠가 하나를 빠뜨립니다. 좁은 통로 3개로 모아 여기서만 검사합니다.

SECURITY

오류 정제는 공통 계층

orchestrator/runner.py 의 스크러버가 오류 메시지의 URL 쿼리스트링을 제거한 뒤 기록합니다. 핸들러가 빠뜨려도 시크릿이 남지 않습니다.

왜: 실제로 토큰이 오류 로그에 섞여 나간 적이 있습니다. "각자 조심"이 아니라 통과 지점에서 일괄 처리합니다.

TECH SPEC

개발 스펙

새 스택을 고르는 대신, 이미 운영 중인 두 플랫폼과 같은 인프라를 썼습니다. 혼자 운영하는 시스템에서 스택이 늘어나면 장애 대응 비용이 그만큼 늘기 때문입니다.

BACKEND

- Python 3.11 · FastAPI
단일 앱 · 라우터 21개
- SQLAlchemy 2.0 (async)
테이블 29개 · Alembic
- Pydantic v2
요청·응답 스키마 계약
- Pillow · ffmpeg
카드뉴스 · 릴스 자체 렌더러
- pytest · ruff · mypy
879 통과 · 린트 · 타입체크

FRONTEND

- React 18 · TypeScript
컴포넌트 162 파일
- Vite
빌드 · 개발 서버
- TanStack Query
서버 상태 · 캐시 무효화
- Zustand
클라이언트 상태
- vitest
73건 통과

INFRA & OPS

- Fly.io (도쿄)
scale-to-zero · release_command
- Neon PostgreSQL
싱가포르 · pooled/direct 분리
- Upstash Redis
레이트리밋 · 멍등키
- Cloudflare Pages · R2
대시보드 호스팅 · 미디어
- GitHub Actions cron
내부 API 호출 오케스트레이션

배치 스케줄

레인	주기	잡
every-100m	100분	publish_due · approval_timeout · watchers · telegram_poll · rollup_ingest_daily · poll_aiwb_daily
every-6h	6시간	collect_trends
nightly	매일 03:00 KST	refresh_tokens · poll_embolos_snapshot · bandit_update · media_privatize
morning	매일 05:00 KST	generate_content · adjust_ads
weekly	월 09:00 KST	weekly_report
manual	수동	잡 이름 지정 실행

SCALE

개발 규모

2026년 8월 3일 첫 커밋부터 8월 24일까지 22일간,
설계·구현·배포·운영을 혼자 수행했습니다.

21,160 LOC

백엔드 애플리케이션
Python 129 파일

17,137 LOC

테스트 코드
프로덕션 코드 대비 81%

16,565 LOC

프론트엔드
TypeScript 162 파일

177 커밋

22일간
일 평균 8건

작업 규율

CI 에 || true 를 쓰지 않습니다. 실패는 실패로 남깁니다. 이전 프로젝트에서 CI 가 실패를 무시하도록 설정돼 회귀를 몇 주 동안 숨긴 적이 있었고, 그때부터 "통과한 것처럼 보이는 파이프라인"을 가장 큰 리스크로 봅니다. 코드 리뷰가 "없는 코드"를 잡지 못한다는 것도 같은 경험에서 배웠습니다 — 그래서 명세와 산출물을 별도로 대조하는 단계를 뒀습니다.

강지호

jihono55@gmail.com

화면 캡처는 2026-08-25 운영 환경 실측입니다.

본문 수치(테스트·LOC·커밋)는 저장소에서 직접 계측한 값입니다.

• Product Engineer · 2026

제품을 만들고, 그 제품을 검증하는 도구까지 만듭니다.

기획·설계·구현·운영을 혼자 끝내는 것을 기준으로 삼습니다. 아래 두 프로젝트는 모두 실제로 돌아가고 실제 산출물을 냅니다 — 이 문서의 화면과 리포트는 전부 실행 결과를 그대로 캡처한 것입니다.

[github.com/StockHedge](#) [jihono55@gmail.com](#) 최근 업데이트 2026.08

2개

단독 개발·운영 제품

10,232줄

TypeScript · Python

60 커밋

두 저장소 추적

47회

실제 테스트 세션 실행

20명

설계한 평가 페르소나

PROJECT 01

AI 베타테스터

출시 전에, 사람 대신 사람처럼 써보는 QA 에이전트. 게임과 웹 서비스를 페르소나 20명의 시선으로 직접 플레이·탐색하고, 근거가 붙은 채점 리포트를 만든다.

Next.js 16

React 19

TypeScript

Tailwind CSS 4

Supabase · Postgres · RLS

Anthropic SDK

Playwright

Zod

Android adb

Docker

npm workspaces

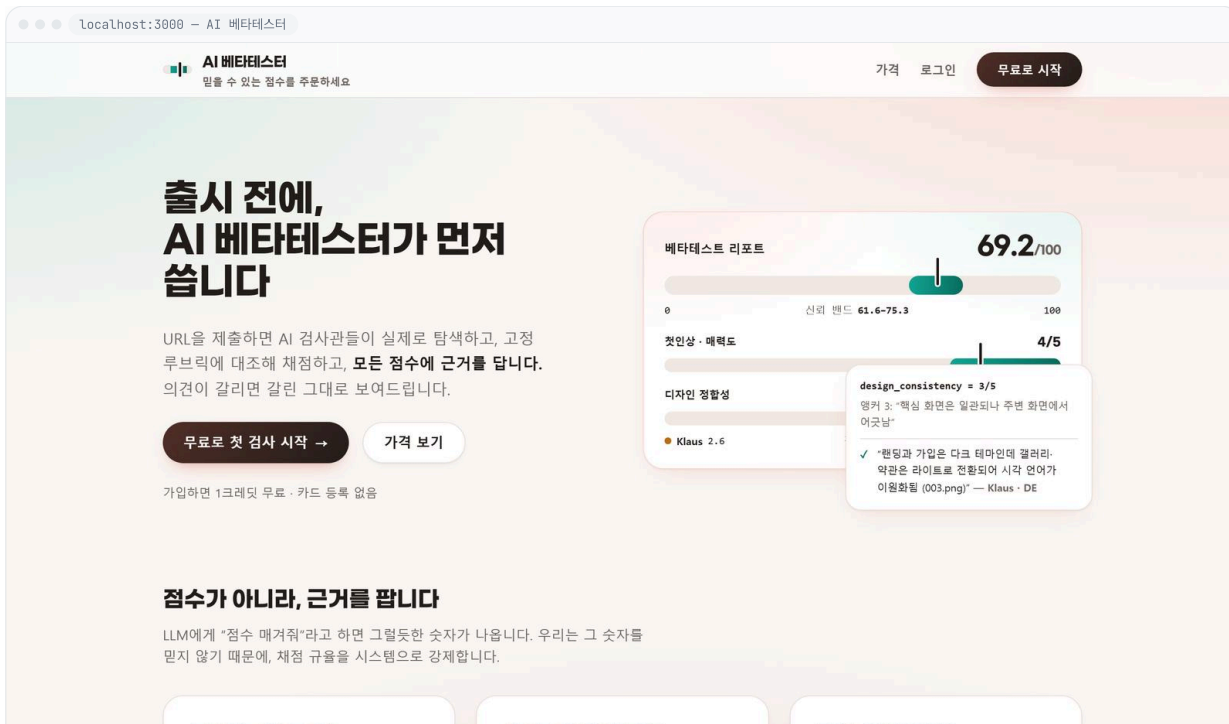
Vercel

PROBLEM

출시 직전 QA는 비싸고 느리다. 자동화 테스트는 "버튼이 눌리는가"는 잡아내지만 "재미가 없다 / 어디를 눌러야 할지 모르겠다"는 잡지 못한다. 그렇다고 실제 베타테스터를 모으면 모집·일정·비용이 전부 걸린다. 결국 사람의 판단이 필요한 영역이 통째로 비어 있는 채로 출시하게 된다.

SOLUTION

나이·국가·성향이 다른 페르소나가 에뮬레이터와 브라우저에서 제품을 직접 조작하고, 축별 앵커 루브릭으로 채점한다. 모든 점수에 관찰 근거를 강제해 "왜 3점인가"가 스크린샷 단위로 추적된다. 결과는 외부 리소스 의존이 0인 자립형 HTML 리포트 한 장이다.



N

모든 축 점수에 실제 관찰(스크린샷-URL-콘솔 오류) 인용이 강제됩니다. 리프트의 전소 연애는 것과같이 실제 한 마디

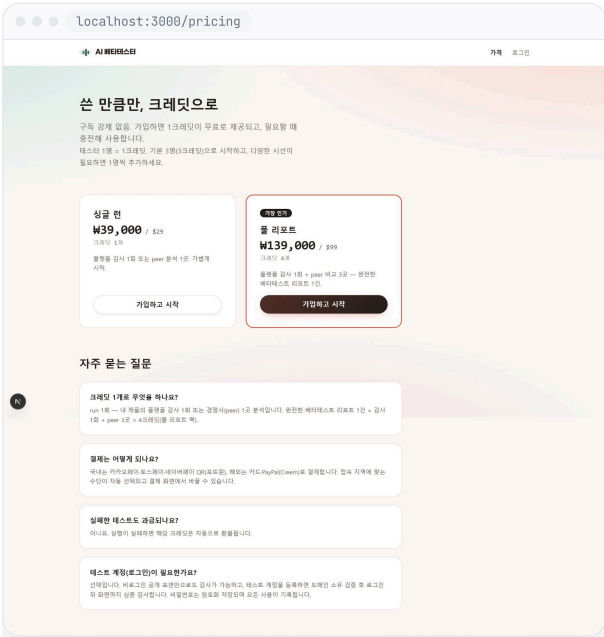
점수는 점이 나와 범위

검사관 여러 명의 판단이 같려면 밴드가 넓어지고, 어느 쪽으로 갈렸는지까지 보여줍니다. 만데는 파다아 포이지 오차가

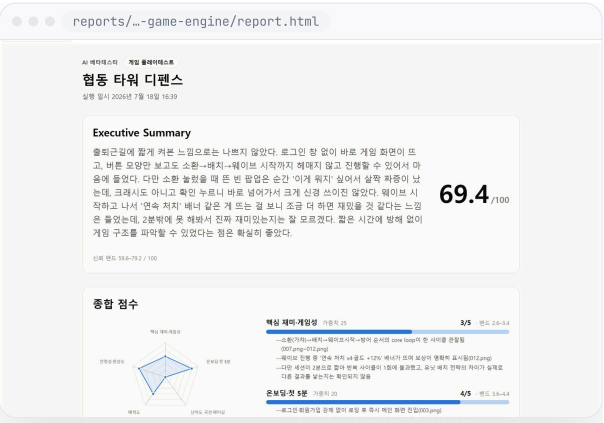
관찰 못 한 축은 N/A

도달하지 못한 화면은 추측으로 채점하지 않고 N/A로 제외한 뒤 가중치를 재분배합니다. 초점이 어떤 커브리지에서

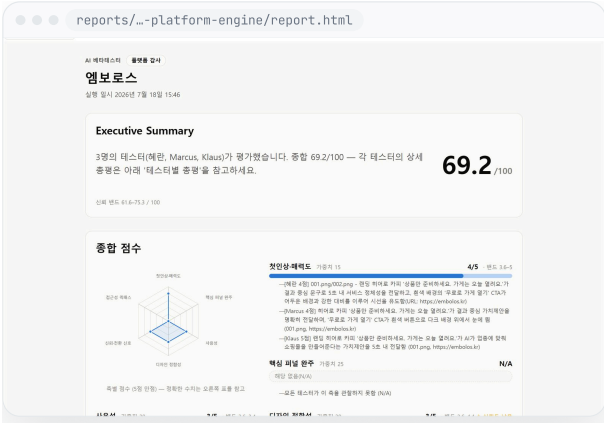
서비스 랜딩 — router.js 16 App Router · React 19 서버 컴포넌트, Supabase 세션 유무를 서버에서 판정해 랜딩과 대시보드로 분기한다.



요금제 — 구독형(로컬 러너)과 종량형(API 엔진), 원가 구조가 다른 두 실행 경로를 각각 상품화했다.



게임 플레이테스트 리포트 — 종합 69.4/100, 신뢰 밴드 59.6–79.2. 각 축 아래에 관찰 근거와 스크린샷 파일명이 붙는다.



플랫폼 UI/UX 감사 — 웹 서비스의 핵심 파일을 브라우저로 탐색하며 이탈 지점을 기록한다.



경쟁사 비교 — 공개 표면만 같은 루브릭으로 훑어 비교 매트릭스를 만든다. 계정 생성·품 제출은 하지 않는다.

설계에서 가장 신경 쓴 것

- 01

근거 없는 점수를 만들지 않는다

LLM 채점의 최대 리스크는 그럴듯한 숫자를 지어내는 것이다. 채점 규율을 별도 모듈로 분리해 관찰 인용이 없는 축은 채점 자체를 하지 않도록 강제했다. 리프트의 모든 점수에는 근거 문장과 012.png 같은 스크린샷 참조가 따라붙는다.
- 03

비용 경로를 돌로 나눴다

API 종량 과금은 세션당 실측 \$0.11–0.16. 여기에 더해 사용자의 기존 구독을 그대로 쓰 는 로컬 러너 경로를 별도로 만들어, 원가 구조가 다른 두 상품을 같은 엔진 위에 올렸다.

- 02

단일 점수 대신 신뢰 밴드

"69.4점"만 보여주면 정말해 보이는 착시가 생긴다. 세션 간 의견 분산을 59.6–79.2 구간으로 함께 노출해 얼마나 확실할 수 있는 점수인지까지 읽히게 했다. 재실행 시 이전 런과의 밴드 diff도 산출한다.
- 04

크롤링 유리를 코드로 강제

경쟁사 분석은 경계가 분명해야 한다. robots.ts로 robots.txt를 준수하고, peer 분석 에이전트는 계정 생성·품 제출 없이 공개 표면만 관찰하도록 권한 자체를 제한했다.

출어진 개발 커뮤니티의 최근 72시간을 3일마다 한국어 리포트로 묶어 PDF로 만들고 카카오톡으로 보내는 무인 파이프라인.

Python 3.11 GitHub Actions Reddit API HN Algolia RSS Kakao OAuth Google Drive API fpdf2 fontTools 서버셋

PROBLEM

바이브 코딩 · Claude Code 관련 정보가 Reddit, Hacker News, 깃뉴스에 흩어져 있다. 매일 다섯 군데를 돌아 훑는 습관은 현실적으로 유지되지 않고, 한 번 놓치면 다시 따라잡을 방법이 없다.

SOLUTION

수집 → 한국어 보고문 조립 → PDF 생성 → Drive 공개 업로드 → 카카오톡 발송을 하나의 무인 루틴으로 묶었다. GitHub Actions에서 돌기 때문에 내 PC가 꺼져 있어도 3일마다 도착한다.

● ● ● reports/digest-2026-08-11.pdf

커뮤니티 인기 글 다이제스트

수집 창: 2026-08-08 11:02 ~ 2026-08-11 11:02 (KST)
총 21건 · 요약 21건

r/ClaudeCode

1. Claude Pro가 없을 때
원제: When you don't have Claude Pro
-- 2026-08-08 20:31
본문 내용이 확보되지 않아 상세한 정보는 알 수 없으나, Claude Pro 구독이 없는 상태에서 겪는 상황에 대해 다루고 있는 것으로 보입니다. 제목을 바탕으로 볼 때, 무료 버전을 사용할 때의 제약이나 경험을 공유하는 글인 것으로 추측됩니다.
https://www.reddit.com/r/ClaudeCode/comments/1vltmsp/when_you_dont_have_claude_pro/

2. 엔트로픽: "코딩은 거의 해결되었습니다"... 또한 엔트로픽: "에이전트가 다른 Claude Code 세션에 메시징을 보낼 수 있는 강력한 새 기능을 출시했지만, 모든 플랫폼에서 기본 기능을 구현할 수 없기 때문에 Linux와 Mac에서만 지원됩니다"
원제: Anthropic: "Coding is largely solved"... Also Anthropic: "We released a new powerful feature allowing agents to send messages to other Claude Code sessions, but only on Linux and Mac since we are unable to implement a basic feature on all platforms"
-- 2026-08-09 21:33
작성자는 엔트로픽이 코딩 문제가 대체로 해결되었다고 주장하면서 정작 기본적인 크로스 플랫폼 기능을 구현하지 못하는 모순을 지적합니다. 작성자는 윈도우 환경에서 이 기능을 오후 반나절 만에 수동으로 코딩하거나 Claude를 통해 단 몇 분 만에 구현할 수 있다고 말합니다. 그럼에도 엔트로픽은 직접 구현하지 않을 뿐만 아니라 Claude Code를 비공개 소스로 유지하여 다른 사람의 참여도 막고 있다고 비판합니다. 결과적으로 대기업의 모순된 태도와 소스 비공개 정책에 대한 강한 불만을 토로하고 있습니다.
https://www.reddit.com/r/ClaudeCode/comments/1vj0q2/anthropic_coding_is_largely_solved_also_anthropic/

3. ADHD 두뇌를 위한 터미널 매니저를 코딩했습니다. 100% 오픈소스입니다.
원제: I coded terminal manager for ADHD brains. 100% Opensource.
-- 2026-08-09 05:01
작성자는 여러 원격 SSH 프로젝트를 관리하고 VS Code를 띄우는 데서 발생하는 비효율을 해결하기 위해 Claude를 사용해 터미널 매니저를 개발했습니다. 이 도구는 단일 캔버스에서 여러 터미널을 관리할 수 있게 도와주며, 기능이 점차 추가되어 자를 개발 환경에 가깝게 발전했습니다. 현재 프로젝트를 계속 추진하기 위해 커뮤니티의 피드백이나 깃허브 스타 등 동기부여와 후원이 필요한 상황입니다. 100% 오픈소스로 개발된 이 도구의 웹사이트와 깃허브 저장소 링크를 공유하여 참여를 독려하고 있습니다.
https://www.reddit.com/r/ClaudeCode/comments/1vj5ktu/i_coded_terminal_manager_for_adhd_brains_100/

r/ClaudeAI

1. 11살 딸이 AI를 사용해 자신만의 브라우저 만들...
원제: When your 11-year-old daughter uses AI to build her own browser...
-- 2026-08-08 22:17
이 글은 11세 딸이 인공지능(AI)을 활용하여 스스로 웹 브라우저 제작한 경험에 관한 것입니다. 본문 내용은 확보되지

자동 생성 PDF · 1면 — 수집한 글을 한국어 보고문으로 조립해 해 최적 저장소에 컷됩니다.

● ● ● reports/digest-2026-08-11.pdf

않아 구체적인 세부 사항은 확인할 수 없으나, AI가 어린아이도 소프트웨어를 개발할 수 있도록 돕는 유용한 도구로 활용되고 있음을 나타냅니다. 제목을 통해 AI 기술의 발전이 개인의 창작 가능성을 넓혀주고 있음을 짐작할 수 있습니다.
https://www.reddit.com/r/ClaudeAI/comments/1vivt77/when_your_11yearold_daughter_uses_ai_to_build_her/

2. 엔트로픽, AI가 위험한 쿼리를 80% 이상 차단하는 반면 인간은 14%만 차단한다는 사실을 발견한 후 8월 14일부터 Claude Code를 기본 자동(Auto) 모드로 전환
원제: Anthropic Flips Claude Code to Auto Mode by Default Aug 14, after finding AI blocks 80%+ dangerous queries while humans only 14%
-- 2026-08-09 12:48
엔트로픽(Anthropic)은 1,053명의 유료 테스터를 대상으로 한 내부 연구 결과를 바탕으로 Claude Code를 기본 자동(Auto) 모드로 전환했습니다. 해당 연구에서 AI 분류기는 위험한 쿼리의 89%를 잡아낸 반면, 동일한 검토를 수행한 인간은 승인 피로감 등으로 인해 탐지율이 13.6%에 불과했으며 50개 프롬프트 이후에는 약 5%로 떨어졌습니다. 또한 자동 모드를 사용하는 팀 및 엔터프라이즈 고객은 약 25% 더 많은 폴 리퀘스트를 처리하는 성과를 보였습니다. 이에 따라 엔트로픽은 분류기가 도구 호출 시 소비하는 소량의 추가 토큰에 대해서는 비용을 청구하지 않겠다고 밝혔습니다.
https://www.reddit.com/r/ClaudeAI/comments/1vj0qv/anthropic_flips_claude_code_to_auto_mode_by/

3. 공익 정보: Claude가 리서치에 WebFetch를 사용하여도록 허용할 때 주의하세요
원제: PSA: Be careful letting Claude use WebFetch for research
-- 2026-08-09 12:42
작성자는 Claude(Opus 5)를 이용해 연구를 진행하던 중, 제공된 데이터와 인용구 중 상당수가 잘못되거나 조작된 것임을 발견했습니다. 분석 결과 WebFetch는 더 작고 저렴한 모델을 사용해 원본을 요약하여 Claude에 전달하는데, 이 과정에서 세부 정보를 왜곡하거나 임의로 지우는 문제가 발생했습니다. 작성자는 WebFetch 사용을 금지하고 서브에이전트가 직접 원본 페이지의 원시 텍스트를 읽도록 워크플로우를 수정하여 오류를 크게 줄이고 연구 품질을 극적으로 개선했습니다. 따라서 Claude가 부정확한 정보를 제공할 때는 소스 데이터가 왜곡되어 전달되는지 확인해 볼 필요가 있다고 조언합니다.
https://www.reddit.com/r/ClaudeAI/comments/1vim8b7/psa_be_careful_letting_claude_use_webfetch_for/

r/vibecoding

1. 지금 내 모습
원제: ME right now
-- 2026-08-10 02:59
개발을 하다 보면 때로는 일이 잘 풀리지 않을 때가 있습니다. 작성자는 지금 현재 자신이 처한 상황을 이 짧은 문장으로 표현하고 있습니다. 비록 지금은 실패했다라도 다음 시도에서는 잘될 것이라고 믿어야 한다는 다짐을 보여줍니다. 바이브 코딩 중 겪는 흔한 좌절과 이를 극복하려는 긍정적인 태도를 나타냅니다.
https://www.reddit.com/r/vibecoding/comments/1vjw12v/me_right_now/

2. 왜 코드가 존재하는 거지?!
원제: Why is there code?!

-- 2026-08-08 17:15
이 글은 코드가 왜 존재하는지에 대해 의문을 제기하는 제목을 가지고 있습니다. 본문 내용이 확보되지 않아 구체적인 맥락은 확인하기 어렵습니다.
https://www.reddit.com/r/vibecoding/comments/1vj0q2/why_is_there_code/

· 2면 — 한글 표시를 위해 Noto Sans KR를 fontTools로 서버셋해 PDF에 임베드했다.

주기 스케줄링의 함정을 코드로 우회한 부분

cron의 */3은 "3일마다"가 아니다. day-of-month 기반이라 31일 달을 지나면 주기가 어긋난다. 그래서 매일 깨우되 epoch-day를 3으로 나눈 나머지로 실행 여부를 결정하는 레이트를 만들어, 월 경계와 무관하게 정확한 3일 간격을 유지했다.

```
# cron은 매일 00:00 UTC(=09:00 KST)에 깨우고,
# epoch-day % 3 게이트로 3일 주기를 만든다.
elif [ $(( $(date +%s) / 86400 % 3 )) -eq 0 ]; then
echo "run=true" >> "$GITHUB_OUTPUT"
```

설계에서 가장 신경 쓴 것

01

의존성을 1개로 묶었다

외부 패키지는 PDF 생성용 fpdf2 하나뿐이고 나머지는 전부 표준 라이브러리다. 무인 루틴에서 의존성은 곧 미래의 장애 지점이라, 편의보다 수명을 택했다.

03

토큰이 만료돼도 멈추지 않게

카카오 refresh token 만료(invalid_grant)는 무인 파이프라인이 조용히 죽는 대표 원인이다. 새 토큰 수신 시 GitHub Secret을 자동 갱신하고, 자동 갱신이 불가하면 재인증 안내를 카카오톡 발송해 사람이 알아채게 만들었다.

02

커뮤니티 단위 실패 격리

수집 대상 하나가 죽어도 다이제스트는 나가야 한다. 수집기를 커뮤니티 단위로 격리해 한 곳의 장애가 전체 발송을 막지 못하게 했다. Reddit 레이트리미트는 잔여량을 감지해 백오프한다.

04

수집 대상을 리서치로 확정

넣을 곳뿐 아니라 제외할 곳과 그 사유까지 문서로 남겼다 — robots 차단, 로그인 벽, 무료 API 부재, ToS상 스크래핑 금지. 나중에 "왜 여긴 없지?"를 다시 조사하지 않기 위한 기록이다.

다루는 것들

두 프로젝트에서 실제로 설계 · 구현 · 운영까지 해본 범위만 적었습니다.

프론트엔드	TypeScript · React 19 · Next.js 16(App Router · 서버 컴포넌트) · Tailwind CSS 4 · WCAG AA 기준 접근성 구현
백엔드 · 데이터	Supabase(Postgres · Auth · RLS) · 마이그레이션 관리 · 서버 전용 키 격리 · Python 3.11
자동화 · 인프라	GitHub Actions(스케줄 · 시크릿 회전) · Docker · Vercel · npm workspaces 모노레포
테스트 · 계속	Playwright(브라우저 자동화 · E2E) · Android adb 기기 제어 · 자체 채점 엔진 설계
AI 응용	Anthropic SDK 기반 에이전트 파이프라인 · 페르소나/루브릭 설계 · 근거 강제 채점 · 실측 기반 원가(COGS) 설계
연동	OAuth 2.0(카카오 · Google) · refresh token 회전 · Reddit / HN Algolia / RSS 수집 · robots.txt 준수

함께 만들 것이 있다면

제품 설계부터 운영까지 혼자 굴러본 경험을 팀에서 쓰고 싶습니다.
저장소는 모두 공개되어 있으니 코드부터 보셔도 좋습니다.

이메일 보내기 — jihono55@gmail.com

GitHub — github.com/StockHedge